claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-a71a55a475829e001.jsonl`
- SHA-256: `5a6da27224349d416e8128d85f05cb5baa3e535f8eeff5469d0ff7d314ec15dd`
- Source modified: `2026-07-08T06:16:00+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_5f13fc79-bad`
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`

Transcript

1. user (2026-07-08T06:12:31.327Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to
an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).
The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):
- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical
  to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the
  SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config
  + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed
  once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing,
  never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a
wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT
report style, naming, or speculative concerns. Cite exact file:line from undefined.

A reviewer claims this DEFECT in the merged #521 code. Try HARD to REFUTE it by reading the actual code at undefined. Default to is_real=false if you cannot concretely confirm the failure. Skeptic #1.

CLAIM (low/robustness) backend/worker/__main__.py:488

summary: cmd_compile writes its done-marker ONLY after _stitch_reel returns; any failure BEFORE
that (spec fetch/parse, time_pairs/stitching coercion, VideoCompiler init) raises out of an
unwrapped main(), so the worker exits with a traceback and NO marker ? unlike cmd_infer's
_fail() which always writes a marker for every worker-side failure.
failure scenario: The bucket-staged spec is truncated/corrupt (transient partial read) or
time_pairs contains a malformed pair. `_read_compile_spec` (worker:426-432) raises
JSONDecodeError/ValueError, or the coercion `[(float(pair[0]), float(pair[1])) for pair ...]`
(worker:493-495) / `StitchingConfig(**...)` (worker:496) raises IndexError/TypeError.
cmd_compile has no surrounding try/except and main() at line 973 calls args.func(args)
unwrapped, so the process exits non-zero with a raw traceback and never writes the --done-uri
marker. The dispatcher's bucket-poll then depends ENTIRELY on the #515 crash-fast Executions-API
probe to convert this into a clean failed job; if that probe is unavailable/delayed, the poll
hangs up to deployment.remote_poll_max_wait_sec (~65 min). cmd_infer avoids this asymmetry ? its
_fail() writes a real-rc marker for every pre-segmentation failure so the dispatcher terminates
fast regardless of the Executions API.

2. assistant (2026-07-08T06:12:38.262Z)

I'll start by locating the actual code, since the paths show "undefined". Let me find the
relevant files.

3. user (2026-07-08T06:12:39.422Z)

backend\worker__main__.py

4. user (2026-07-08T06:12:40.038Z)

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)

5. assistant (2026-07-08T06:12:44.538Z)

The merged code is at HEAD (commit dac6736 = #521 via #525). Let me read the relevant files.

6. user (2026-07-08T06:12:45.770Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
```

```
    """

    from __future__ import annotations

    import argparse
    import csv
    import json
    import logging
    import os
    import re
    import sys
    import tempfile
    from typing import Any, List

    from backend import storage
    from backend.compute import ComputeJobSpec, get_compute_backend
    from backend.config import (
        StartupCheckError,
        enforce_startup_checks,
        load_config,
        probe_cuda_available,
    )
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure
    from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
    from backend.utils.hashing import compute_video_id

    logger = logging.getLogger("backend.worker")


    def _coerce(v: str) -> Any:
        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
        low = v.lower()
        if low in ("true", "false"):
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
```

```
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow(
                    [
                        f"{float(s.get('start_time', 0.0)):.3f}",
                        f"{float(s.get('end_time', 0.0)):.3f}",
                        s.get("shot_count", ""),
                        s.get("ending_reason", s.get("shot_count_confidence", "")),
                    ]
                )


    def _write_report_json(
        video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
    ) -> None:
        with open(path, "w") as f:
            json.dump(
                {
                    "video_id": video_id,
                    "status": status,
                    "segment_count": len(segments),
                    # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
                    # control plane re-hydrating from outputs/<md5>/ restores the video row
                    # with its truthful path ? without this, only the segments recover.
                    "video_uri": video_uri,
                    "segments": [
                        {
                            "start": float(s.get("start_time", 0.0)),
                            "end": float(s.get("end_time", 0.0)),
                        }
                        for s in segments
                    ],
                },
                f,
                indent=2,
            )


    def _serialize_and_push_outputs(
        scratch: str,
        out_uri: str,
        video_id: str,
        segments: List[dict],
        status: str,
        storage_cfg: dict,
        video_uri: str = "",
    ) -> List[str]:
        """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
        ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.

        Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
        ``status="failed"`` report.json in the bucket ? the control plane's source of truth
        (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
        report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
        EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(
```

```
            video_id,
            segments,
            status,
            os.path.join(out_local, "report.json"),
            video_uri=video_uri,
        )
        out_prefix = out_uri.rstrip("/")
        pushed: List[str] = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)
        return pushed


    def _run_startup_checks(config: Any) -> bool:
        """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
        best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
        cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
        (returns True, ZERO work) when no deployment.profile is selected.

        The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
        torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
        WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
        no-op of work, not just of output."""
        profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
        cuda = (
            probe_cuda_available() if profile else None
        )  # torch-free on the default-OFF path
        try:
            enforce_startup_checks(config, cuda_available=cuda, logger=logger)
            return True
        except StartupCheckError:
            return False  # already logged at ERROR by enforce_startup_checks


    def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
        """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
        outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
        container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.

        A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
        from the local 2/3), never a raw traceback:
          * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be
            running (the shim best-effort cancels it).
          * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
            crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
            ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
        from backend.compute import RemoteExecutionFailed
        from backend.infrastructure.execution_policy import PolicyTimeout

        spec = ComputeJobSpec(
```

```
            video_uri=args.video_uri,
            out_uri=args.out_uri,
            segmenter=args.segmenter,
            config_overrides=tuple(args.set or ()),
            skip_validation=bool(getattr(args, "skip_validation", False)),
        )
        try:
            result = compute.run_infer(spec)
        except RemoteExecutionFailed as e:
            logger.error(
                "[worker] remote execution terminated without a completion marker: %s", e
            )
            return 5
        except PolicyTimeout as e:
            logger.warning(
                "[worker] remote compute did not complete (no marker within budget): %s", e
            )
            return 4
        logger.info(
            "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
            result.video_id,
            result.returncode,
            result.outputs_uri,
        )
        return result.returncode


    def _write_done_marker(
        done_uri: str,
        video_id: str,
        returncode: int,
        segment_count: int,
        status: str,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "segment_count": segment_count,
                "status": status,
            }
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
```

```
through to the
252         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
253         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
254         compute = get_compute_backend(config)
255         if compute is not None:
256             return _dispatch_remote(compute, args)
257
258         storage_cfg = config.storage.model_dump()
259
260         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261         os.makedirs(scratch, exist_ok=True)
262         config.output.output_dir = scratch
263
264         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265         local_video = storage.fetch(
266             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267         )
268         print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271         video_id = compute_video_id(local_video)
272
273         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282         if getattr(args, "skip_validation", False):
283             print(
284                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285                 "gate and already validated this input."
286             )
287             val_results, val_context = [], {}
288         else:
289             val_results, val_context = validator_registry.run_all_with_context(
290                 local_video, config
291             )
292         failures = [r for r in val_results if not r.passed]
293         db = Database(os.path.join(scratch, config.output.db_name))
294         db.add_video(
295             video_id,
296             local_video,
297             "failed" if failures else "processed",
298             [r.model_dump() for r in val_results],
299         )
300
301         def _fail(rc: int) -> int:
302             # A worker-side FAILURE (validation, unknown segmenter, or a detector
```

```
timeout/abort the
303             # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
304             # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
305             # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
306             # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
307             # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
308             # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
309             try:
310                 _serialize_and_push_outputs(
311                     scratch,
312                     args.out_uri,
313                     video_id,
314                     [],
315                     "failed",
316                     storage_cfg,
317                     str(getattr(args, "video_uri", "") or ""),
318                 )
319             except Exception as e:  # never mask the real failure on an artifact-push hiccup
320                 logger.warning("[worker] could not push failure artifacts: %s", e)
321             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
322             if getattr(args, "done_uri", None):
323                 _write_done_marker(
324                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
325                 )
326             return rc
327
328         segments: List[dict] = []
329         segmenter_actual = None
330         segmentation_ran = False
331         status = "failed"
332         try:
333             if failures:
334                 print(
335                     "[worker] validation FAILED: "
336                     + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
337                 )
338                 return _fail(2)
339             seg_name = args.segmenter or config.indexing.default_segmenter
340             segmenter_cls = segmenter_registry.get(seg_name)
341             if not segmenter_cls:
342                 print(
343                     f"[worker] unknown segmenter: {seg_name!r} "
344                     f"(have {list(segmenter_registry.list_available())})"
345                 )
346                 return _fail(2)
347             segmenter_actual = seg_name
348             result = segmenter_cls(db, config).process_video(
349                 video_id, local_video, max_frames=args.max_frames
350             )
351             segmentation_ran = True
352             if isinstance(result, Failure):
353                 print(f"[worker] segmenter FAILED: {result.message}")
354                 return _fail(3)
355             segments = result.value if hasattr(result, "value") else result
356             status = "success"
357             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
```

```
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=args.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=False,
                segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        pushed = _serialize_and_push_outputs(
            scratch,
            args.out_uri,
            video_id,
            segments,
            status,
            storage_cfg,
            str(getattr(args, "video_uri", "") or ""),
        )

        print(
            f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
        )
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
```

```
        # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
        # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(
                args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
            )
        return 0


    def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
        """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
``time_pairs``, the
        resolved ``stitching`` config, the source ``video_uri``, ``video_id``,
``output_name`` + ``locale``.
        A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
        local = storage.fetch(
            spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
        )
        with open(local, encoding="utf-8") as f:
            data = json.load(f)
        if not isinstance(data, dict):
            raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
        return data


    def _write_compile_marker(
        done_uri: str,
        result: dict,
        video_id: str,
        returncode: int,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
        dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "status": result.get("status", "failed"),
                "download_url": result.get("download_url", ""),
                "reel_uri": result.get("reel_uri", ""),
                "message": result.get("message", ""),
            }
            local = os.path.join(scratch, "_compile_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote compile marker -> %s", done_uri)
        except Exception as e:  # never fail a good stitch because the marker push hiccuped
            logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)


    def cmd_compile(args: argparse.Namespace) -> int:
        """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
stitch is the
        2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
        the control-plane-staged compile-spec (resolved time-pairs + stitching config +
```

```
source URI), stitch
        the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
there is no
        forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
        bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
        from backend import orchestration
        from backend.config import StitchingConfig

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
        os.makedirs(scratch, exist_ok=True)
        # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
the bucket.
        config.output.output_dir = scratch
        # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
value as a
        # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket
        # destination even if the override were dropped.
        if not (config.storage.output_root or "").strip():
            config.storage.output_root = args.out_uri

        spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
        video_id = str(spec.get("video_id") or "")
        out_name = str(spec.get("output_name") or "")
        source_ref = str(spec.get("video_uri") or "")
        locale = spec.get("locale")
        time_pairs = [
            (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
        ]
        stitching = StitchingConfig(**(spec.get("stitching") or {}))

        print(
            f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
            f"encoder={stitching.encoder} out={out_name}"
        )

        result = orchestration._stitch_reel(
            config,
            stitching,
            video_id,
            source_ref,
            time_pairs,
            out_name,
            locale,
            lambda stage: print(f"[worker] compile: {stage}"),
        )
        status = str(result.get("status") or "failed")
        rc = 0 if status == "success" else 3
        if status != "success":
            print(f"[worker] compile FAILED: {result.get('message')}")

        # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
        if getattr(args, "done_uri", None):
            _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
        return rc


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
```

```
so a
525     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
526     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
527
528     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
529     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
530     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
531     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
532
533
534     def _label_clip_name(fname: str) -> str:
535         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
536
537         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
538         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
539
540         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
541         * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
542         * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
543         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
544
545         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
546         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
547         """
548         name = fname.replace(".rallies.csv", "").replace(".csv", "")
549         return _LABEL_DATE_PREFIX.sub("", name)
550
551
552     def _probe_video_fps_width(path: str):
553         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
554
555         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
556         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
557         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
558         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
559         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
560         rather than guesses.
561         """
562         import json
563         import subprocess
564
565         try:
566             out = subprocess.check_output(
567                 [
568                     ffprobe_bin(),
569                     "-v",
570                     "error",
571                     "-select_streams",
572                     "v:0",
```

```
                    "-show_entries",
                    "stream=r_frame_rate,width",
                    "-of",
                    "json",
                    path,
                ],
                stderr=subprocess.DEVNULL,
            ).decode()
            st = (json.loads(out).get("streams") or [{}])[0]
            num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
            fps = float(num) / float(den or "1")
            width = float(st.get("width") or 0)
            if fps > 0 and width > 0:
                return fps, width
        except (
            subprocess.SubprocessError,
            ValueError,
            KeyError,
            OSError,
            json.JSONDecodeError,
        ):
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(
                f"[worker] unknown segmenter: {args.segmenter!r} "
                f"(have {list(segmenter_registry.list_available())})"
            )
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(
                f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
            )
            return None
        try:
            runner, _ = seg._resolve_runner(config.indexing)
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
```

```
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(
            u
            for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
            if u.lower().endswith(".csv")
        )
        if len(label_uris) < 2:
            print(
                f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                f"under {corpus}/labels/"
            )
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                if not vname.lower().endswith(_VIDEO_EXTS):
                    continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
                video_by_stem[os.path.splitext(vname)[0]] = vuri

```

```
        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name  # e.g. 2026-06-22_GX020094.rallies.csv
            name = _label_clip_name(fname)  # -> GX020094 (date prefix stripped)
            local_label = storage.fetch(
                uri, os.path.join(labels_dir, fname), config=storage_cfg
            )
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(
                        f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
                    )
                    continue
                local_video = storage.fetch(
                    vuri,
                    os.path.join(videos_dir, storage.parse_uri(vuri).name),
                    config=storage_cfg,
                )
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(
                        f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
                    )
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(
                        f"[worker] detector produced no trajectory for {name!r} ? skipping."
                    )
                    continue
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": fps,
                        "frame_width": frame_width,
                    }
                )
            else:
                from backend.pipeline.detectors.stub_runner import (
                    synth_trajectory_from_labels,
                )

                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(
                    local_label, traj, fps=args.fps, frame_width=args.frame_width
                )
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": args.fps,
                        "frame_width": args.frame_width,
                    }
```

```
                )

        if len(specs) < 2:
            print(
                f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
            )
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = (
            "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
        )
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [
            sys.executable,
            "-m",
            "training.gen0.harness",
            "--manifest",
            manifest_path,
            "--out",
            out_dir,
            "--version",
            args.version,
        ]
        if args.floor:
            floor_local = (
                storage.fetch(
                    args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
                )
                if "://" in args.floor
                else args.floor
            )
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            prog="python -m backend.worker",
            description="Storage-aware worker: pull ? run pipeline ? push.",
        )
        p.add_argument(
            "-v",
            "--verbose",
```

```
            action="store_true",
            help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
        )
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument(
            "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
        )
        pi.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (outputs/<video_id>/? is appended)",
        )
        pi.add_argument(
            "--segmenter", default=None, help="default: indexing.default_segmenter"
        )
        pi.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pi.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument(
            "--done-uri",
            default=None,
            help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
            "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
            "Default-OFF: omit it for the normal local run.",
        )
        pi.add_argument(
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set indexing.skip_ai_handoff=true",
        )
        pi.add_argument(
            "--skip-validation",
            action="store_true",
            help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
            "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
            "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
            "`worker infer` CLI still validates.",
        )
        pi.set_defaults(func=cmd_infer)

        pc = sub.add_parser(
            "compile",
            help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged
compile-spec",
        )
        pc.add_argument(
            "--spec-uri",
            required=True,
            help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
stitching "
            "config + source uri + video_id)",
        )
        pc.add_argument(
```

```
            "--out-uri",
            default="",
            help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
            "Also forwarded via --set storage.output_root; either resolves the reel path.",
        )
        pc.add_argument(
            "--done-uri",
            default=None,
            help="storage URI for the compile completion MARKER (rc + download_url), so a
remote "
            "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI
stitch.",
        )
        pc.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pc.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pc.add_argument(
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set storage.output_root=gs://bucket",
        )
        pc.set_defaults(func=cmd_compile)

        pt = sub.add_parser(
            "train",
            help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
        )
        pt.add_argument(
            "--corpus-uri",
            required=True,
            help="prefix containing labels/ (+ videos/ for "
            "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
        )
        pt.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (weights/<version>/ is appended)",
        )
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument(
            "--trajectory-source",
            choices=["synth", "detector"],
            default="synth",
            help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
            "detector = REAL shuttle trajectories via the resolved DetectorRunner "
            "(needs videos/ in the corpus; honours detector_impl).",
        )
        pt.add_argument(
            "--segmenter",
            default="wasb_hybrid",
            help="trajectory hybrid whose detector is used for --trajectory-source
detector",
        )
        pt.add_argument(
            "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
        )
        pt.add_argument("--config", default=None)
        pt.add_argument("--scratch", default=None)
        pt.add_argument(
```

```
937             "--fps",
938             type=float,
939             default=30.0,
940             help="synth trajectory fps (detector probes the video)",
941         )
942         pt.add_argument(
943             "--frame-width",
944             type=float,
945             default=1280.0,
946             help="synth trajectory frame width (detector probes the video)",
947         )
948         pt.add_argument("--set", action="append", default=[], metavar="k=v")
949         pt.set_defaults(func=cmd_train)
950         return p
951
952
953     def _setup_logging(verbose: bool) -> None:
954         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
955         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
956         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
957         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
958         entry), not at import, so importing the module never reconfigures a caller's
logging."""
959         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
960         # configured logging at import), which would leave the level at WARNING and re-
suppress the
961         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
962         logging.basicConfig(
963             level=logging.DEBUG if verbose else logging.INFO,
964             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
965             datefmt="%H:%M:%S",
966             force=True,
967         )
968
969
970     def main(argv=None) -> int:
971         args = build_parser().parse_args(argv)
972         _setup_logging(getattr(args, "verbose", False))
973         return args.func(args)
974
975
976     if __name__ == "__main__":
977         sys.exit(main())
978
```

7. assistant (2026-07-08T06:13:26.460Z)

Now let me examine the dispatch side and the #515 crash-fast mechanism. Let me read the orchestration compile dispatch and the cloud_run backend.

8. user (2026-07-08T06:13:29.116Z)

```
1:"""Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of #390).
4:Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` / ``_ingest_remote_segments``),
5:and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
56:# #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg compile
stitch ?
60:# while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no deadlock;
```

```
when the
122:    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
produced.
170:# guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic deep
220:    ``_maybe_dispatch_remote`` decision WITHOUT side effects (#521), so
``_execute_processing`` can
433:_MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
450:def _maybe_dispatch_remote(
463:    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
608:        # DETECTs from it, and a later /api/compile re-fetches the same source (the local
upload is
820:    # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
862:        # clip via a non-zero rc, which _maybe_dispatch_remote surfaces as a failed job.
When detection
966:        # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
970:        remote = _maybe_dispatch_remote(
1091:    """A submit-time-validatable compile problem ? carries the HTTP status + detail so the
API path
1100:def _resolve_compile(
1103:    """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
1135:            "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
1150:    """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
1151:    the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
1194:def _compile_wait_message(elapsed_sec: float) -> str:
1195:    """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
1203:def _stitch_reel(
1213:    """Stitch + mirror-upload core, shared by the control-plane compile handler AND the L4
worker
1214:    (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1233:            "message": f"Unsupported compile rendition: {rendition}",
1239:        video, kept, out_name = _resolve_compile(
1254:    compiler = _main.VideoCompiler(
1268:        "[compile] stitching video_id=%s rendition=%s output=%s",
1282:        # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1284:            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1287:            "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1313:            logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1333:def _dispatch_compile_remote(
1342:    """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-plane.
The
1343:    control-plane owns the DB, so it RESOLVES the reel's segments here
(``_resolve_compile``), stages a
1344:    small compile-spec (resolved time-pairs + the resolved stitching/watermark config + the
source URI)
1345:    to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-polls
the
1348:    ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
1353:        video, kept, out_name = _resolve_compile(
1368:            "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
1373:        return _stitch_reel(
1377:    # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe on
the wire);
1395:    spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
1397:        notify("staging compile spec")
1398:        with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
1405:            "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
```

```
exc_info=True
1409:            "message": f"cloud compile: could not stage the compile spec: {e}",
1414:    # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
1429:        "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d
clip(s))",
1435:        result = compute.run_compile(
1436:            spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
1439:        logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
1442:            "message": f"cloud compile dispatch error: {e}",
1448:            or f"cloud compile job failed (returncode {result.returncode})."
1477:def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1478:    """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
1483:    #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``) and
cloud
1485:    (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
1486:    2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process here,
unchanged."""
1498:        return _dispatch_compile_remote(
1504:        video, kept, out_name = _resolve_compile(
1512:    return _stitch_reel(
```

9. user (2026-07-08T06:13:30.948Z)

```
7:  2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage layer (D16
? the
8:     completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
11:     job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-but-
healthy
14:     *Fast negative path:* while the marker stays the SOLE success signal, each poll also
probes the
15:     Cloud Run Executions API ? if the execution has TERMINATED without ever writing a marker
(a
16:     worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17:     (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
18:  3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
44:    RemoteExecutionFailed,
95:        terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails fast
instead
96:        of bucket-polling the full budget. The done-marker stays the ONLY success signal.
99:        chooses not to ? poll the Executions API degrades transparently to today's marker-
only
176:        # None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never raises
on bad
236:        # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
245:    def run_infer(
253:        done_uri = f"{out_root}/_dispatch/{token}.done"
261:            done_uri,
279:            done_uri,
282:        marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-infer")
283:        video_id = str(marker.get("video_id", ""))
285:        # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
286:        rc = int(marker.get("returncode", 1))
289:                "cloud-run: job=%s completion marker present but empty/unreadable (no "
307:    def run_compile(
314:        bucket-poll the done-marker, and return the worker's rc + marker. Reuses the EXACT
315:        dispatch?poll?marker machinery as ``run_infer`` (``_await_marker`` ? crash-fast +
timeout
```

```
320:        # Distinct marker prefix from infer's _dispatch/ so a compile + a detection for the
same
321:        # bucket can never collide on a marker.
322:        done_uri = f"{out_root}/_compile/{token}.done"
330:            done_uri,
344:            done_uri,
346:        marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-compile")
347:        video_id = str(marker.get("video_id", "") or spec.video_id)
348:        rc = int(marker.get("returncode", 1))
355:        # The marker carries the worker's stitch result (download_url / reel_uri / status);
surface it
360:            outputs_uri=str(marker.get("reel_uri", "") or ""),
362:            report=marker,
366:    def _await_marker(
368:        done_uri: str,
374:        """Bucket-poll ``done_uri`` to a completion marker ? shared by ``run_infer`` and
376:        caller so job.progress moves). A ``RemoteExecutionFailed`` (the fast negative path
in
377:        ``_poll_check``) is NOT a ``PolicyTimeout`` so it propagates uncaught ? the dispatch
handler
381:                lambda: self._poll_check(done_uri, str(execution or "")),
388:            return self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
390:    def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
393:        The done-marker is the SOLE success signal and is checked FIRST ? a present marker
always
395:        FAILURE* marker resolves through the normal path with its real returncode. Only when
the
396:        marker is absent do we consult the execution status: a worker that TERMINATED
without ever
397:        writing a marker (crash / OOM / an old image argparse-aborting on an unknown flag)
would
398:        otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min) before
failing
402:        Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
403:        ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over, no
marker
405:        marker = self._read_marker(done_uri)
406:        if marker is not None:
407:            return marker  # success signal ? authoritative even if status later flips
411:        # The execution is terminal but no marker is present. Re-check the marker ONCE to
close the
413:        # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
415:        marker = self._read_marker(done_uri)
416:        if marker is not None:
417:            return marker
418:        raise RemoteExecutionFailed(
420:            f"terminal state {state!r} with NO completion marker ? the worker exited before
"
421:            "writing one (crash / OOM / pre-marker abort, or a failed marker upload on an "
422:            "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
430:        flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
432:        ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
445:    def _handle_poll_timeout(
446:        self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
452:        ONE final marker check rescues that completed GPU run instead of wasting it. If the
marker
458:            late = self._read_marker(done_uri)
461:                "cloud-run: final post-timeout marker check failed ? treating as absent",
467:                "cloud-run: job=%s exec=%s done-marker found on the final post-timeout check
"
493:    def _read_marker(self, done_uri: str) -> Optional[dict]:
494:        """The done-marker dict if the worker has written it, else None ? poll_until keeps
```

```
waiting."""
495:        ref = parse_uri(done_uri)
499:            self._read_json(done_uri) or {}
```

10. assistant (2026-07-08T06:14:01.930Z)

Let me read the `_stitch_reel` core and the spec-building dispatch to confirm the stitch-failure handling and how the spec is produced.

11. user (2026-07-08T06:14:03.465Z)

```
1203    def _stitch_reel(
1204        cfg,
1205        stitching,
1206        video_id: str,
1207        source_ref: str,
1208        time_pairs: List[Tuple[float, float]],
1209        out_name: str,
1210        locale: Optional[str],
1211        notify,
1212    ) -> Dict[str, Any]:
1213        """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1214        (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1215        needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1216        with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1217        trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
1218        returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1219        ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
1220        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
1221
1222    <<<<<<< Updated upstream
1223    =======
1224        notify = progress or (lambda stage: None)
1225        params = json.loads(job.get("params") or "{}")
1226        video_id = job.get("video_id") or ""
1227        segment_ids = params.get("segment_ids") or []
1228        locale = params.get("locale")
1229        rendition = str(params.get("rendition") or "original")
1230        if rendition not in {"original", "sd"}:
1231            return {
1232                "status": "failed",
1233                "message": f"Unsupported compile rendition: {rendition}",
1234                "video_id": video_id,
1235            }
1236
1237        notify("resolving")
1238        try:
1239            video, kept, out_name = _resolve_compile(
1240                db, cfg, video_id, segment_ids, params.get("output_filename", "")
1241            )
1242        except _CompileError as e:
1243            return {"status": "failed", "message": e.detail, "video_id": video_id}
1244
1245        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1246    >>>>>>> Stashed changes
1247        output_dir = (
1248            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
```

```
        )
        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
        # unknown/absent locale keeps the configured default unchanged.
        localized_watermark = localize_watermark(stitching.watermark, locale)
        encode_profile = _rendition_encode_profile(stitching, rendition)
        compiler = _main.VideoCompiler(
            output_dir,
            begin_padding=stitching.begin_padding,
            end_padding=stitching.end_padding,
            watermark=localized_watermark,
    <<<<<<< Updated upstream
            encoder=getattr(stitching, "encoder", "libx264"),
    =======
            encode_profile=encode_profile,
    >>>>>>> Stashed changes
        )

        notify("stitching")
        logger.info(
            "[compile] stitching video_id=%s rendition=%s output=%s",
            video_id,
            rendition,
            out_name,
        )
        local_src = ""
        try:
            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
            local_src = storage.acquire_local_input(
                source_ref, getattr(cfg, "storage", None)
            )
            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
            with _LOCAL_COMPUTE_LANE:
                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
            logger.error(
                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
            )
            return {
                "status": "failed",
                "message": f"Highlight stitching failed: {e}",
                "video_id": video_id,
            }
        finally:
            if local_src:
                storage.cleanup_acquired_local_input(
                    source_ref, local_src, getattr(cfg, "storage", None)
                )

        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
        # still falls back to the local file on the same instance). On the L4 worker the
```

```
mirror is how the
1304        # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
1305        reel_uri = reel_object_uri(cfg, video_id, out_name)
1306        if reel_uri:
1307            try:
1308                notify("uploading reel")
1309                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1310                # project/client) is used, not a fresh default.
1311                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1312            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1313                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1314
1315        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1316        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1317        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1318        notify("finished")
1319        return {
1320            "status": "success",
1321            "filepath": out_path,
1322            "download_url": download_url,
1323            "video_id": video_id,
1324    <<<<<<< Updated upstream
1325            "reel_uri": reel_uri or "",
1326    =======
1327            "filename": out_name,
1328            "rendition": rendition,
1329    >>>>>>> Stashed changes
1330        }
1331
1332
```

12. assistant (2026-07-08T06:14:12.902Z)

I notice unresolved merge-conflict markers in the working tree's orchestration.py. Let me verify whether I'm reading the actual merged commit or a dirty working tree.

13. user (2026-07-08T06:14:15.855Z)

```
M  README.md
M  SESSION_HANDOFF.md
M  backend/config/models.py
M  backend/infrastructure/database_jobs.py
M  backend/main.py
UU backend/orchestration.py
UU backend/pipeline/stitcher/compiler.py
M  config.json
M  tests/test_api_endpoints.py
M  tests/test_async_jobs.py
M  tests/test_compiler.py
M  tests/test_watermark.py
?? score_golden_gate.py
?? score_local.py
---HEAD worker diff check---
 backend/orchestration.py | 54 ++++++++++++++++++++++++++++++++++++++++++++++++++
 1 file changed, 54 insertions(+)
```

14. assistant (2026-07-08T06:14:24.652Z)

The working tree has an in-progress merge with conflicts (unrelated WIP from another session). The worker file `__main__.py` is clean (matches HEAD), so my reading of it is valid. But I must

```
read orchestration.py from the committed HEAD, not the conflicted working tree. Let me read the
committed versions.
```

15. user (2026-07-08T06:14:26.889Z)

```
    with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source, runs
the
    trim/watermark/concat, mirrors the reel to ``<output_root>/highlights/<video_id>/<name>``,
and
    returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls for.
Returns a
    ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
    import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main

    output_dir = (
        getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
    )
    # Localize the branding watermark to the UI locale the SPA rendered in (recorded on the
job);
    # unknown/absent locale keeps the configured default unchanged.
    localized_watermark = localize_watermark(stitching.watermark, locale)
    compiler = _main.VideoCompiler(
        output_dir,
        begin_padding=stitching.begin_padding,
        end_padding=stitching.end_padding,
        watermark=localized_watermark,
        encoder=getattr(stitching, "encoder", "libx264"),
    )

    notify("stitching")
    local_src = ""
    try:
        # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local path).
Resolve to
        # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-scratch for a
URI (the
        # same seam the process path uses); without it, compiling a bucket-ingested video can't
open gs://.
        local_src = storage.acquire_local_input(
            source_ref, getattr(cfg, "storage", None)
        )
        # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count (for
remote
        # dispatch) doesn't run multiple compiles at once on the box (see _LOCAL_COMPUTE_LANE).
        with _LOCAL_COMPUTE_LANE:
            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
    except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job FAILURE, not
a crash
        logger.error(
            "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
        )
        return {
            "status": "failed",
            "message": f"Highlight stitching failed: {e}",
            "video_id": video_id,
        }
    finally:
        if local_src:
            storage.cleanup_acquired_local_input(
                source_ref, local_src, getattr(cfg, "storage", None)
            )

    # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed URL ?
```

```
Cloud Run
    # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp copy dies
when the
    # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch (the
endpoint
    # still falls back to the local file on the same instance). On the L4 worker the mirror is
how the
    # reel reaches the bucket at all (the worker's local file is discarded when the Job exits).
    reel_uri = reel_object_uri(cfg, video_id, out_name)
    if reel_uri:
        try:
            notify("uploading reel")
            # Thread the active storage settings so the configured backend (S3 endpoint/region,
GCS
            # project/client) is used, not a fresh default.
            storage.put(out_path, reel_uri, config=_storage_settings(cfg))
        except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch already
succeeded
            logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)

    # `filepath` is server-local (not browser-reachable); also return the download URL so the
SPA needn't
    # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
    download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
    notify("finished")
    return {
        "status": "success",
        "filepath": out_path,
        "download_url": download_url,
        "video_id": video_id,
        "reel_uri": reel_uri or "",
    }


def _dispatch_compile_remote(
    cfg,
    db,
    video_id: str,
    segment_ids: List[int],
    output_filename: str,
    locale: Optional[str],
    notify,
) -> Dict[str, Any]:
    """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-plane. The
    control-plane owns the DB, so it RESOLVES the reel's segments here (``_resolve_compile``),
stages a
    small compile-spec (resolved time-pairs + the resolved stitching/watermark config + the
source URI)
    to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-polls the
    done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
expects. The
    worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
    ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
    from backend.compute import CompileJobSpec, get_compute_backend

    notify("resolving")
    try:
        video, kept, out_name = _resolve_compile(
            db, cfg, video_id, segment_ids, output_filename
        )
    except _CompileError as e:
        return {"status": "failed", "message": e.detail, "video_id": video_id}

    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
    storage_cfg = getattr(cfg, "storage", None)
```

```
    out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
    compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else None
    if compute is None:
        # Compile was placed on the L4 but the cloud dispatch isn't actually wired here (no
output
        # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ? a slow
reel
        # beats no reel. (The placement guard means this is only reachable in a mis-provisioned
setup.)
        logger.warning(
            "[compile] compile placed on the L4 but no cloud backend available (out_root=%r) ? "
            "stitching in-process on the control-plane instead.",
            out_root,
        )
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
        return _stitch_reel(
            cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale, notify
        )

    # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe on the
wire);
    # the stitching config is the control-plane's RESOLVED one, with the encoder swapped to the
L4's
    # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes the
reel.
    stitch_dump = stitching.model_dump()
    stitch_dump["encoder"] = getattr(
        getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
    )
    spec_payload = {
        "video_id": video_id,
        "video_uri": video["filepath"],
        "output_name": out_name,
        "locale": locale,
        "time_pairs": [
            [float(s["start_time"]), float(s["end_time"])] for s in kept
        ],
        "stitching": stitch_dump,
    }
    token = uuid.uuid4().hex
```

16. assistant (2026-07-08T06:15:00.257Z)

```
I have everything I need. Let me confirm the dispatcher wraps `run_compile` in a try/except (the
last link in the chain).
```

17. user (2026-07-08T06:15:03.251Z)

```
    the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
    result for an EXPECTED problem (missing video / no segments / a stitch error) so the SPA
always gets
    a clean verdict instead of a worker-crash error.

    #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``) and cloud
    dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
    (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip reel on
the
    2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process here,
unchanged."""
    notify = progress or (lambda stage: None)
    params = json.loads(job.get("params") or "{}")
    video_id = job.get("video_id") or ""
    segment_ids = params.get("segment_ids") or []
    locale = params.get("locale")
```

```
    output_filename = params.get("output_filename", "")

    if (
        resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
        and _cloud_available(cfg)
    ):
        return _dispatch_compile_remote(
            cfg, db, video_id, segment_ids, output_filename, locale, notify
        )

    notify("resolving")
    try:
        video, kept, out_name = _resolve_compile(
            db, cfg, video_id, segment_ids, output_filename
        )
    except _CompileError as e:
        return {"status": "failed", "message": e.detail, "video_id": video_id}

    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
    time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
    return _stitch_reel(
        cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale, notify
    )
```

18. user (2026-07-08T06:15:08.081Z)

```
    notify("finished")
    return {
        "status": "success",
        "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
        "download_url": download_url,
        "video_id": video_id,
        "compute": {
            "path": "cloud_run",
            "execution": result.execution,
            "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
            "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
        },
    }


def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
    """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the reel,
and return
    the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
    result for an EXPECTED problem (missing video / no segments / a stitch error) so the SPA
always gets
    a clean verdict instead of a worker-crash error.

    #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``) and cloud
    dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
    (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip reel on
the
    2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process here,
unchanged."""
    notify = progress or (lambda stage: None)
    params = json.loads(job.get("params") or "{}")
    video_id = job.get("video_id") or ""
    segment_ids = params.get("segment_ids") or []
    locale = params.get("locale")
    output_filename = params.get("output_filename", "")

    if (
        resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
```

```
        and _cloud_available(cfg)
    ):
        return _dispatch_compile_remote(
            cfg, db, video_id, segment_ids, output_filename, locale, notify
        )

    notify("resolving")
    try:
        video, kept, out_name = _resolve_compile(
            db, cfg, video_id, segment_ids, output_filename
        )
    except _CompileError as e:
        return {"status": "failed", "message": e.detail, "video_id": video_id}

    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
    time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
    return _stitch_reel(
        cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale, notify
    )
```

19. user (2026-07-08T06:15:12.617Z)

```
    #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``) and cloud
    dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
    (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip reel on
the
    2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process here,
unchanged."""
    notify = progress or (lambda stage: None)
    params = json.loads(job.get("params") or "{}")
    video_id = job.get("video_id") or ""
    segment_ids = params.get("segment_ids") or []
    locale = params.get("locale")
    output_filename = params.get("output_filename", "")

    if (
        resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
        and _cloud_available(cfg)
    ):
        return _dispatch_compile_remote(
            cfg, db, video_id, segment_ids, output_filename, locale, notify
        )

    notify("resolving")
    try:
        video, kept, out_name = _resolve_compile(
            db, cfg, video_id, segment_ids, output_filename
        )
    except _CompileError as e:
        return {"status": "failed", "message": e.detail, "video_id": video_id}

    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
    time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
    return _stitch_reel(
        cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale, notify
    )
```

20. user (2026-07-08T06:15:18.807Z)

```
107:    except Exception:
260:    except Exception:  # noqa: BLE001 ? best-effort identity, the drain still hashes
340:    except Exception:  # noqa: BLE001 ? a failed probe degrades to the guarded dispatch path
416:    except Exception:  # noqa: BLE001 ? cache misses must degrade to a normal dispatch
429:    except Exception:  # noqa: BLE001 ? recovery nicety, never a failure
595:        except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job failure,
```

not a 500
685:    except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
708:    except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not 500
or leak rows
795:    except Exception:
802:    except Exception:
901:            except Exception: # noqa: BLE001 ? any cache read/replay fault degrades to a
real validation
988:        except Exception:
1083:   except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must not
block
1236:   except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1263:   except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch already
succeeded
1349:   except Exception as e: # noqa: BLE001 ? a staging failure is a clean job failure, not
a crash
1381:   result = compute.run_compile(
1384:   except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
1385:   logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
1388:   "message": f"cloud compile dispatch error: {e}",

21. assistant (2026-07-08T06:15:49.938Z)

I have fully traced the chain in the committed HEAD (the merged #521) code. Here is my
assessment.

Refutation verified in code:

1. **The claimed asymmetry with `cmd_infer` is factually false.** `cmd_infer`'s `_fail()` is
only defined at `__main__.py:301` and covers failures *after* config-load, `_apply_overrides`,
startup-checks, the input-video `storage.fetch`, and `compute_video_id`. Specifically:
   - `_run_startup_checks` failure ? `return 2` at line 248, **no marker written**.
   - `local_video = storage.fetch(args.video_uri, ...)` at lines 265-267 raises on a
truncated/failed fetch *before* `_fail` is even defined ? this is the exact analog of
`cmd_compile`'s `_read_compile_spec` at line 488. A corrupt/partial fetch of the input in
`cmd_infer` *also* escapes `main()` with a traceback and no marker.

   So `cmd_infer` does **not** "write a real-rc marker for every pre-segmentation failure." Both
commands leave early fetch/parse failures marker-less and rely on the identical downstream
safety net. #521 introduces no asymmetry.

2. **The marker-less termination is caught by the deliberate, shared #515 machinery.**
`cloud_run.py::run_compile` (line 307) reuses the exact `_await_marker` ? `_poll_check` path as
`run_infer`. `_poll_check` (lines 390-423) raises `RemoteExecutionFailed` precisely when an
execution reaches a terminal state with no marker ? its own docstring lists "crash / OOM / an
old image **argparse-aborting** on an unknown flag." A traceback-exit from a corrupt spec is
exactly that class.

3. **The job-level invariant holds.** In `_dispatch_compile_remote`, `compute.run_compile(...)`
(orchestration HEAD line 1381) is wrapped in `except Exception` (line 1384) that returns
`{"status": "failed", "message": "cloud compile dispatch error: ?", "video_id": ?}`. So a
marker-less worker crash surfaces to the SPA as a clean failed job with the polled shape ? "a
stitch failure must be a clean failed-job, never a crash" is preserved at the level the SPA
observes. `_stitch_reel` itself independently wraps the actual stitch in try/except (HEAD line
1236) and returns a clean failed dict.

4. **The residual concern is speculative and pre-existing.** The claim's own fallback ("if the
#515 probe is unavailable/delayed, the poll hangs up to ~65 min") applies equally to
`cmd_infer`'s early failures and to any worker crash ? the class #515 was purpose-built for.
Even in that degraded case the marker-only path terminates cleanly at `PolicyTimeout` (rc 4, a
failed job), never an infinite hang or a wrong result. The staged spec is written by the
control-plane then read back over a consistent object store; "transient partial read" of a small

JSON is not a demonstrated real failure mode.

The claim correctly notes `cmd_compile` lacks a try/except around spec-fetch/coercion, but it does not produce a wrong result, a job-level crash, or silent data loss beyond what the intended, shared design already converts into a clean failed job ? and its central premise (that `cmd_infer` is asymmetric) is contradicted by `__main__.py:265` and `:248`.

(Note: the working tree currently has an unrelated in-progress merge with conflict markers in `orchestration.py`/`compiler.py` from another session; I reviewed the committed HEAD `dac6736` via `git show`, and `backend/worker/__main__.py` is clean and matches HEAD.)

22. user (2026-07-08T06:16:00.441Z)

Structured output provided successfully